

ARCHITETTURE ORIENTATE AI SERVIZI

e

WEB SERVICES

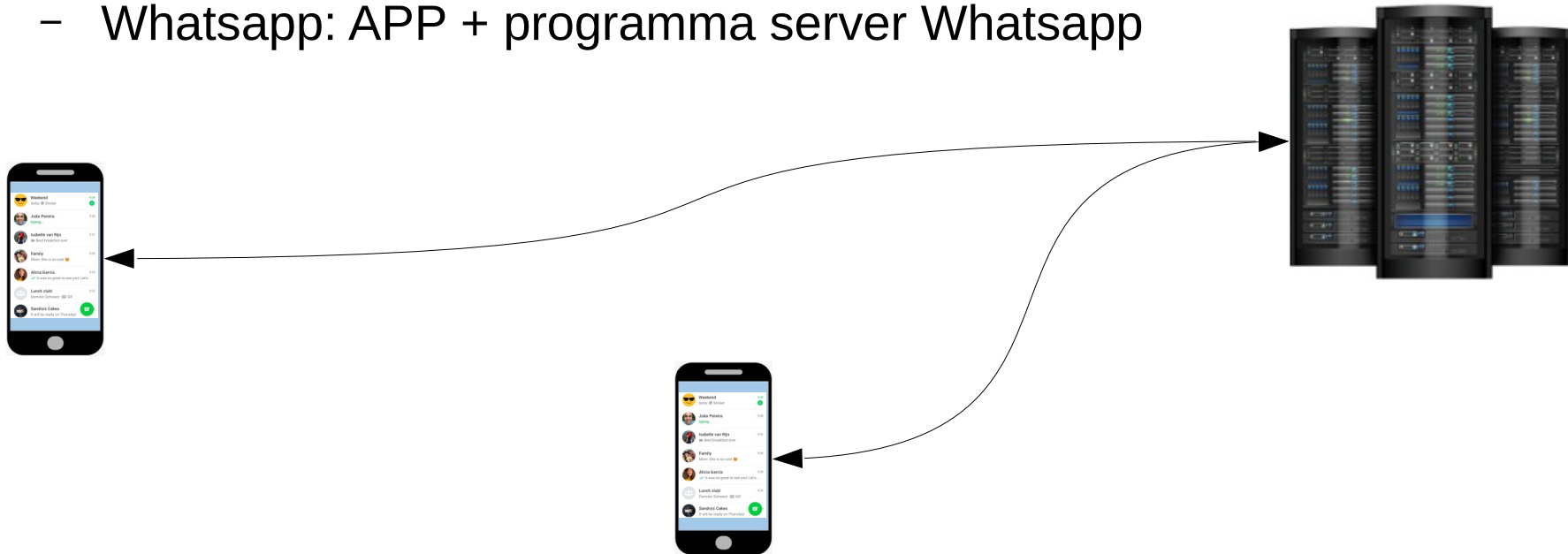
Scrittura di applicazioni che usano la rete

- Programmi in esecuzione su host diversi che comunicano tra loro usando la rete, ad es:
 - Web: browser + programma sul sito (server) web



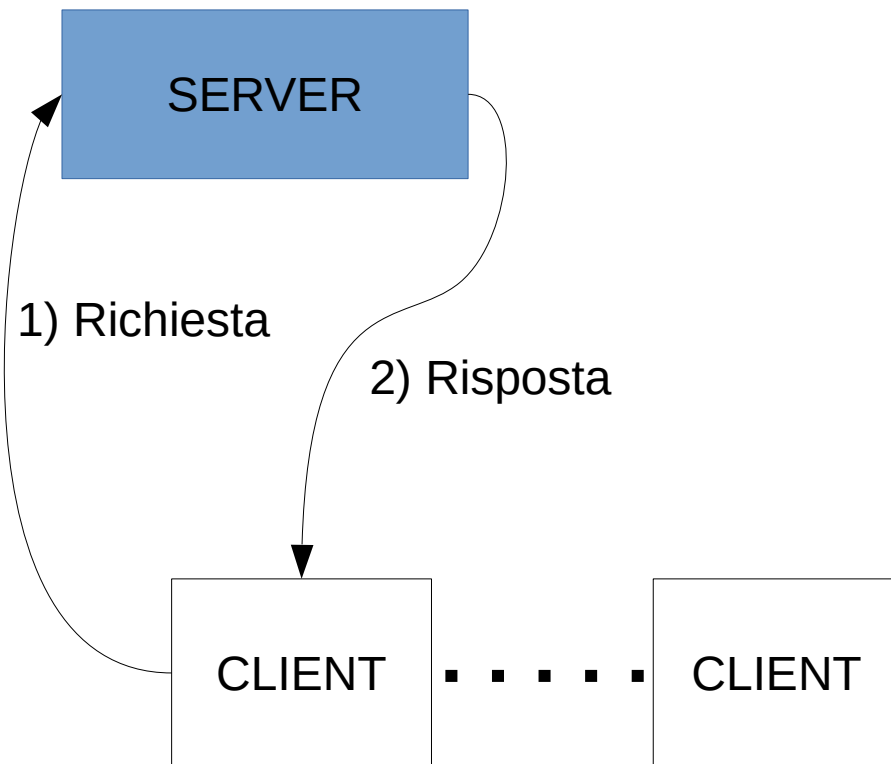
Scrittura di applicazioni che usano la rete (cont.)

- Programmi in esecuzione su host diversi che comunicano tra loro usando la rete, ad es:
 - Whatsapp: APP + programma server Whatsapp



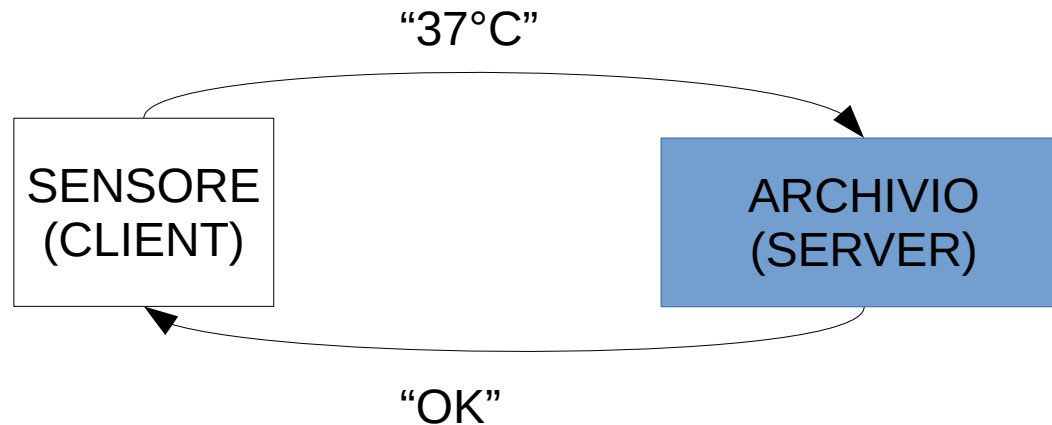
Modello client/server

- E' la situazione più frequente
- Il client fa sempre il primo passo con una **richiesta**
- Il server manda la **risposta** e poi si rimette in attesa di altre richieste
- NOTA: La richiesta del client può essere una richiesta di un dato oppure la trasmissione di un dato (cioè la richiesta di prendere in consegna un certo dato)



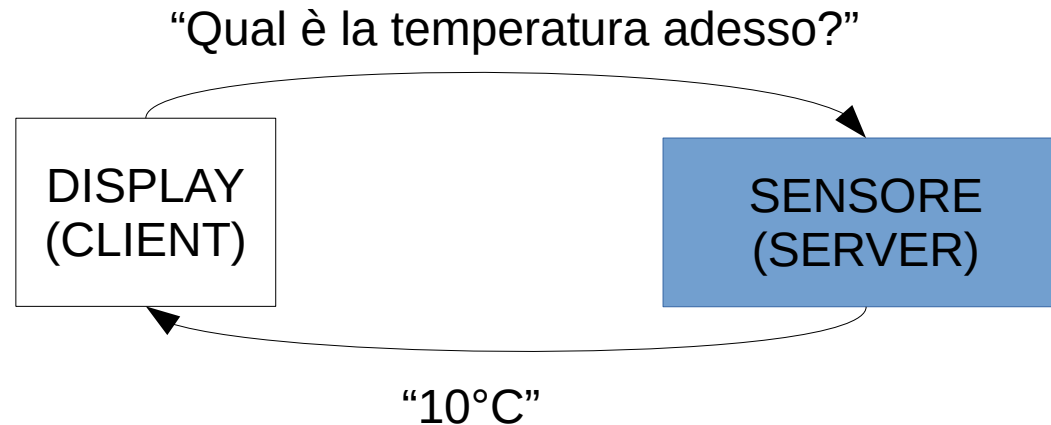
Esempio di applicazione client/server

- Il sensore di temperatura corporea funge da client e manda al server una temperatura
- Il server risponde con un “OK”



Altro esempio di applicazione client/server

- Il display funge da client e chiede al server una temperatura
- Il server risponde con il dato



Architetture orientate ai servizi

Service-oriented architecture (SOA)

- Tradizionali applicazioni “monolitiche”:
 - l’interfaccia utente richiama delle **funzionalità** fornite da una serie di **librerie** linkate in un unico programma che “gira” sulla macchina dell’utente
- SOA: realizzazione di applicazioni complesse attraverso la **combinazione di diversi programmi attraverso la rete**
 - L’interfaccia utente + qualche funzionalità di base “girano” sull’host dell’utente
 - Le funzionalità principali dell’applicazione sono fornite da programmi che girano su uno o più server

SOA: vantaggi

- Potenza di calcolo e memoria sono delegate ai server
- Protezione della proprietà intellettuale su algoritmi strategici
- Annullamento della necessità di distribuire aggiornamenti del software quando le modifiche riguardano solo il codice dei server
- Nuovo modello economico: pay per use
- Eliminazione della pirateria

SOA: requisiti

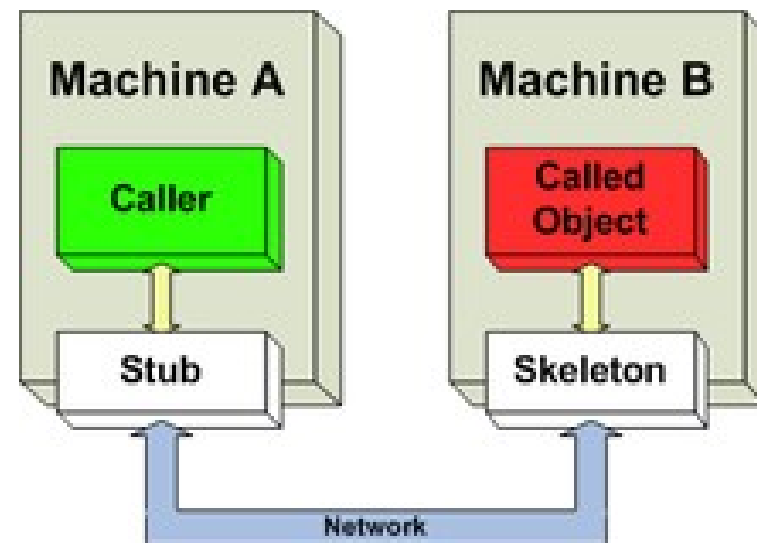
- L'infrastruttura di rete diventa un elemento essenziale
- Se “manca la rete” l'applicazione non funziona

SOA: tecnologie

- **Servizio** = funzionalità = chiamata a funzione = metodo (per gli ambienti orientati agli oggetti)
 - È il cuore di SOA
- Componenti di una funzione:
 - Nome → ci dice cosa fa la funzione
 - Tipo e numero di parametri in ingresso
 - Tipo di valore restituito
 - Implementazione
- **Interfaccia** = descrizione di nome, parametri e valore di ritorno delle funzioni erogate da una libreria che è locale in caso di applicazioni tradizionali oppure remota in caso di SOA
- Application program interface (**API**) = insieme delle funzioni/metodi esposte da una certa libreria locale o da un server remoto
- Il cuore di SOA è la **chiamata di funzione remota** perfettamente conforme al modello client/server

Chiamata di funzione remota

- Il componente server espone una API che descrive una serie di funzioni che il componente client (che **NON è un web browser**) può invocare
- L'implementazione delle funzioni sta nel server
- Il codice che chiama la funzione sul client e quello che implementa la funzione sul server possono essere scritti con linguaggi differenti e girare su architetture di calcolo molto differenti
 - Entrambe le funzioni implementano la **STESSA INTERFACCIA**
- La funzione chiamata dal client apparentemente realizza la funzionalità; in realtà CODIFICA i parametri per essere trasmessi in rete e DE-CODIFICA il valore di ritorno
 - Tale funzione si chiama **STUB**
- Sul server esiste un componente chiamato **SKELETON** che DE-CODIFICA i parametri di input, li passa alla funzione che contiene l'implementazione vera e propria (**BUSINESS LOGIC**), prende il risultato, lo CODIFICA e lo spedisce al client
- Ci vuole un protocollo di rete per il trasporto dei dati codificati
 - La codifica a volte si definisce SERIALIZZAZIONE



Tecnologie/standard per la chiamata di funzione remota

- Remote procedure call (RPC)
 - C su TCP
- Java Remote Method Invocation (JAVA RMI)
 - Java su TCP
- Common Object Request Broker Architecture (CORBA)
 - standard indipendente dai linguaggi di programmazione e dal protocollo di liv. trasporto
- **Webservice**
 - HTTP/HTTPS come protocollo di trasporto degli elementi della funzione
 - Formato dei dati
 - Protocollo XML SOAP: pesante e datato
 - Metodologia **REST** (attualmente la più usata)

Webservice basati su REST

- Utilizzo di HTTP/HTTPS come protocollo che veicola la chiamata
- Mapping del **nome della funzione** sulla URL
- Passaggio dei **parametri** sulla URL (metodo GET o DELETE) oppure dopo l'header (metodo POST o PUT)
- Scelta del metodo HTTP in funzione della **semantica** della funzione
 - POST: usato per funzioni che creano un NUOVO oggetto sul server
 - PUT: funzioni che aggiornano un oggetto ESISTENTE sul server
 - GET: funzioni che recuperano info di un oggetto ESISTENTE sul server
 - DELETE: funzioni che distruggono un oggetto esistente sul server
- **Valore di ritorno** nel corpo della risposta
 - Sostituzione di HTML con
 - Testo puro
 - JSON

Metodi nella richiesta HTTP

- GET
- POST
- PUT
- DELETE
- HEAD
- CONNECT
- OPTIONS
- TRACE
- PATCH

Webservice: vantaggi

- L'infrastruttura Internet è già predisposta all'uso di HTTP/HTTPS
 - Firewall
 - NAT
- L'uso di contenuti testuali nelle transazioni facilita il debugging delle applicazioni SOA
- Lo stesso insieme di servizi può essere richiamato sia da un programma client sia da un browser web
 - Esempio: webservice associati ad una applicazione di Internet Banking usati sia da un sito web sia dall'APP.

JSON

- Formato di dato testuale nato con Javascript ma oggi supportato in tutti i linguaggi di programmazione
- Struttura gerarchica facilmente leggibile da un umano e parserizzabile da un programma
- Elemento base: coppia attributo : valore
- Tipi base: stringa (da indicare sempre tra ""), numero, boolean, null
- Composizione di elementi omogenei: array [...]
- Composizione di elementi eterogenei: struttura dati {...}

Esempi di formattazione JSON

```
{"nome": "Ugo", "età": 30, "laurea": null, "abbonato": false}
```

```
{"impiegati": [  
  { "nome": "Giovanni", "cognome": "Rossi" },  
  { "nome": "Anna", "cognome": "Bianchi" },  
  { "nome": "Pietro", "cognome": "Verdi" }  
]}
```

Esercizio

- Considerare la cartella `Webservice/`
- Aprire il file `serverHTTP-REST.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il file `clientREST-GET.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
 - Che parametri devo passare in linea di comando?
 - Cosa si può vedere analizzando lo scambio di dati tramite Wireshark?
 - Qual è la signature della funzione `calcolaSomma()` sul server e sul client?
Perché ha senso che siano uguali?

Esercizio 2

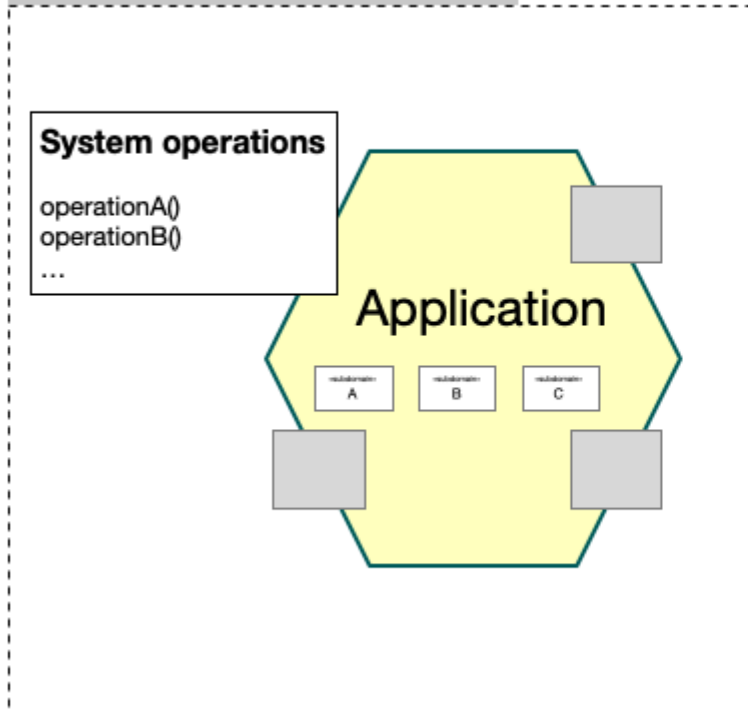
- Prendere in considerazione il file `ClientREST.java`
 - Dopo aver installato l'ambiente base di Java (già presente in Lab Delta) si può compilare con `javac ClientREST.java`
 - Ed eseguire con `java ClientREST`
- Il fatto che il server sia fatto in C e il client in Java è un problema o un'opportunità? Perché?

Esercizio 3

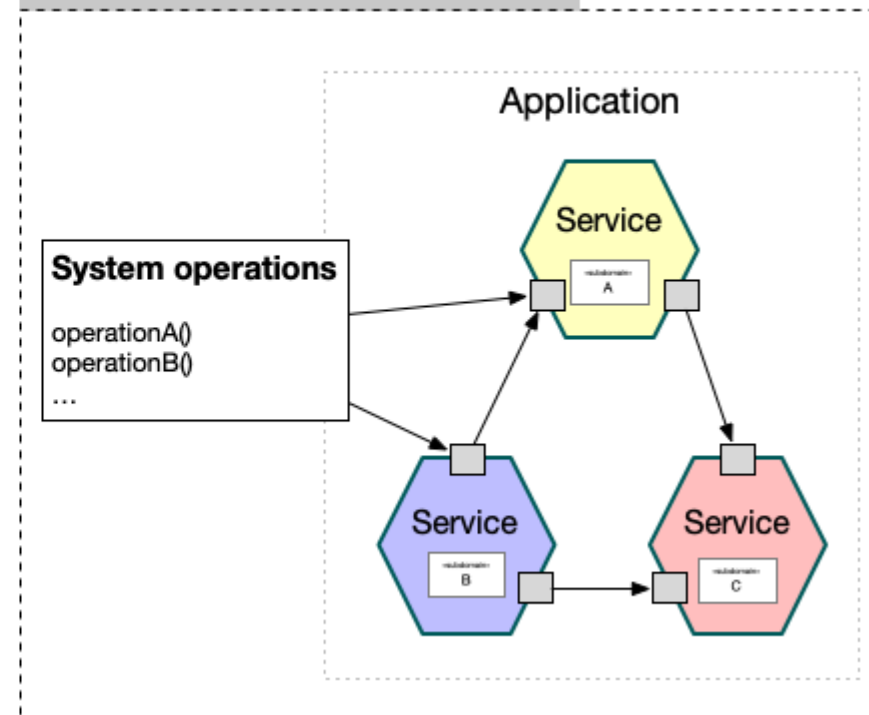
- Estendere il webservice `serverHTTP-REST.c` in modo che esponga un secondo servizio relativo al calcolo dei numeri primi compresi nell'intervallo `[min, max]`
 - Si tragga spunto al programma `prime-number-interval.c`
- Estendere il file `ClientREST.java` in modo da poter chiamare, a scelta, entrambe le funzionalità della nuova API
- Provare il client con il calcolo dei numeri primi compresi nell'intervallo `[1, 1000000]`
 - Quanto tempo ci mette?
 - E' stato necessario tradurre l'algoritmo dei numeri primi in Java? Perché?

Architettura SOA a microservizi

Monolithic architecture



Microservice architecture



Pro e contro dei microservizi

- Posso rendere modulare la parte di backend
 - assegnare moduli software a macchine differenti per aumentare la scalabilità a fronte di aumento di richieste dei client (vedere il concetto di LOAD BALANCING più avanti)
 - maggiore facilità di sviluppo e manutenzione
 - programmare interventi di manutenzione e sviluppo su certi moduli lasciando attivi gli altri
 - ridurre il rischio di effetti collaterali
- Se suddivido le tabelle della base dati su più database differenti si elimina la possibilità di JOIN ottimizzato dei DBMS relazionali e si deve implementare manualmente l'incrocio delle tabelle
 - occorre dividere la base date in punti dove l'esigenza di incrocio è minima

Esercizio finale

- Prendere in considerazione il file `clientThreadREST.java`
 - Esso invoca `calcolaSomma()` in 3 thread concorrenti
 - Però la macchina su cui gira il server chiamato è la stessa e quindi non ci guadagno in prestazioni
 - Come si potrebbe modificare il codice in modo che le 3 invocazioni finiscano su 3 macchine diverse?
 - Bisogna modificare anche il codice del server?
- Si riconsideri consideri il servizio che calcola i numeri primi nell'intervallo `[min, max]` costruito nell'esercizio precedente
 - Si trovi un modo efficiente, sfruttando diversi server in rete, per calcolare il numeri primi tra 1 e 1000000
 - Di quanto migliorano le prestazioni?
 - Devo modificare anche il codice del server?

Progettazione del backend

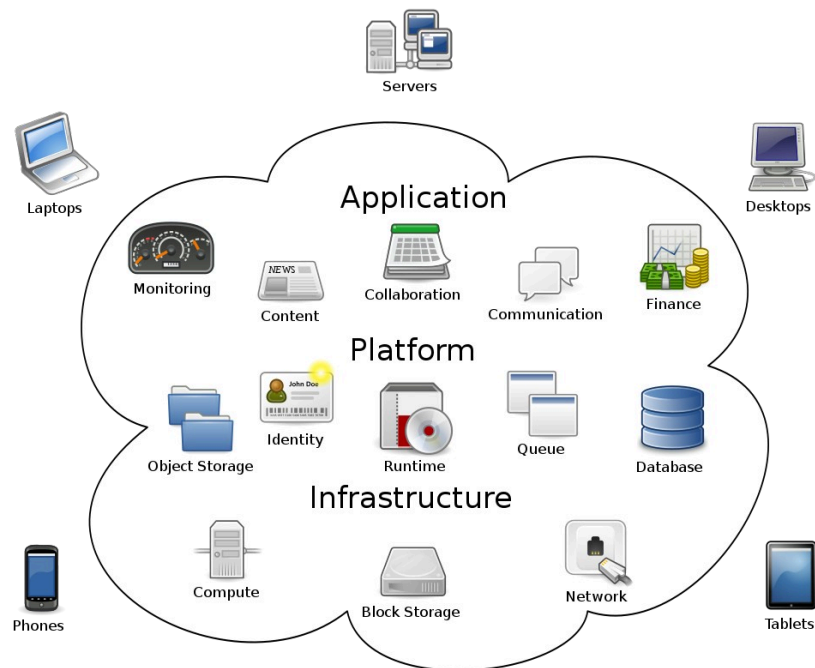
- Definire l'insieme dei servizi in cui partizionare l'applicazione
- Per ciascun servizio:
 - decidere il metodo HTTP tra GET, POST, PUT, DELETE
 - decidere la forma dell'URL con relativi parametri
 - definire il formato del JSON restituito
- Implementare i servizi con le librerie software necessarie per creare i server (Apache, NodeJS, Flask, ecc...)
- Istanziare gli applicativi server sulle varie macchine
- Ottimizzare l'esecuzione dei server sulle varie macchine

Ottimizzazione del backend

- Posso istanziare più server dello stesso tipo e distribuire le richieste degli utenti sulle varie istanze → LOAD BALANCING
 - la containerizzazione (es. Docker) permette di replicare facilmente le istanze
 - L'orchestrazione (es. Kubernetes) permette di variare dinamicamente il numero di istanze in funzione del carico e di riavviare istanze in crash
- Diversi modi per fare load balancing
 - servizi diversi vanno su istanze diverse (non è un vero e proprio load balancing)
 - presenza di un server unico con IP corrispondente al nome di host presente nelle varie URL con il solo ruolo di smistare le richieste alle varie istanze che stanno dietro
 - può tenere conto dell'identità dell'utente e mandare richieste successive dello stesso utente sulla stessa istanza
 - il DNS risolve il nome dell'host presente nelle URL fornendo IP diversi corrispondenti alle diverse istanze
 - Problema del caching delle informazioni del DNS nella rete del client

Cloud computing

- Gruppo di host in rete che forniscono servizi di calcolo e memorizzazione senza essere indirizzati o gestiti individualmente dagli utenti
 - Amazon Web Service
 - Google Cloud
 - Microsoft Azure
 - Dropbox
- L'intero insieme di hardware e software è gestito dal fornitore del cloud che si fa pagare con una politica a consumo oppure forfettaria
- Tecnologie abilitanti per il cloud:
 - Utenti perennemente connessi in rete
 - Virtualizzazione per ottimizzare l'uso delle risorse HW
 - Webservice per sfruttare le risorse di calcolo da remoto



Cloud: servizi offerti

- **On-site** (oppure **on-premise**): l'hardware è mio e si trova a casa mia; devo gestire tutto io.
- **Infrastructure as a Service (IaaS)**: affitto una macchina virtuale "vuota" su cui installo il mio sistema operativo e sopra tutte le mie applicazioni.
- **Platform as a Service (PaaS)**: affitto l'uso di un ambiente in cui c'è già il S.O. e posso installare direttamente le mie applicazioni (ad es. con Docker)
- **Software as a Service (SaaS)**: affitto l'uso di applicazioni già installate, ad es.
 - Web server
 - Database server
 - Broker per pub/sub

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

■ You manage
■ Service provider manages

Sistema di containerizzazione (ad es. Docker) Fonte: www.redhat.com

Function-as-a-Service (FaaS)

- Nuova offerta cloud simile a SaaS
- Possibilità di creare direttamente delle **funzioni**
 - Scegliendo il linguaggio preferito
 - Vengono associate a degli eventi
 - Vengono eseguite allocando al volo le risorse di calcolo necessarie
 - Il committente non deve creare neanche il processo server che le fornisce
- Servizi commerciali
 - AWS Lambda.
 - Google Cloud Functions.
 - IBM Cloud Functions based on Apache OpenWhisk.
 - Microsoft Azure Functions.
 - Oracle Cloud Functions.

Function-as-a-Service (FaaS)

```
void resizeImage(Image img)
{
    resize(img);
    ...
}
```

- Granularità di suddivisione del calcolo ancora più fine rispetto all'architettura a microservizi
- anche conosciuto come “serverless computing” anche se questo è un nome poco significativo

